

When The Numbers Are Wrong

Nothing failed. Nothing alerted. The total is thirty percent too high.

Why this is not normal debugging

- ◆ **There is no stack trace.** Nothing threw. The query succeeded.
- ◆ **There is no failing test.** The output is well formed and in range.
- ◆ **You often cannot see the right answer.** Only that this one looks wrong.
- ◆ What you have instead is counts, and counts are enough.

THE DIFFERENCE

Debugging application code narrows by exception. Debugging data narrows by arithmetic.

Two ways to ask for help

THE DEFAULT

Describe the symptom

- Paste the query
- Say the total is too high
- Ask what is wrong with it
- Get four plausible theories

WHAT WE BUILD

Bisect with counts

- Count at every stage
- Find where the number changes
- Prove which join causes it
- One answer, with evidence

THE TIEBREAKER

Four theories feels like progress. It is four things to rule out, and it produced no evidence.

Do not ask what is wrong. Ask for the queries that will **tell** you.

What you actually type

You want queries back, not opinions.

The query below returns a total that is roughly 30% too high.
It runs without error. Do not tell me what you think is wrong.

Write a single diagnostic query that returns one row per stage
of this query's construction, with the row count at that stage.

Stages, in order:

1. driving table with its own filters only
2. after each join is added, one at a time, in query order
3. after the WHERE clause

Use UNION ALL with a label column so I can read it top down.

Then state which stage you expect the count to change at, and
why. I will run it and tell you the actual numbers.

THE LINES THAT MATTER

The last two lines force a prediction. A wrong prediction beats a vague theory.

The instrument, not the answer

Run it. Read down the labels.

```
01 SELECT '1 driving'      AS stage, COUNT(*) FROM wauf w
02   WHERE w.wotyp IN ('C1','C2','C3')
03 UNION ALL
04 SELECT '2 plus wopr',    COUNT(*) FROM wauf w
05   JOIN wopr o ON o.wonum = w.wonum
06   WHERE w.wotyp IN ('C1','C2','C3')
07 UNION ALL
08 SELECT '3 plus t401',    COUNT(*) FROM wauf w
09   JOIN wopr o ON o.wonum = w.wonum
10   JOIN t401 t ON t.wotyp = w.wotyp
11   WHERE w.wotyp IN ('C1','C2','C3')
```

THE RESULT

Predicted: no change at stage three. Actual: the count tripled. That is the whole investigation.

Reading the counts

- ◆ **Count unchanged.** That join is one to one. Move on.
- ◆ **Count increased.** Fan-out. The next prompt tells you which key is at fault.
- ◆ **Count decreased.** An inner join is dropping rows. Often the real bug.
- ◆ **Count matches the prediction.** Good. Keep going, and stay suspicious.

THE POINT

A decrease is the finding people miss. Nobody investigates a total that is too low.

You know which join. Now find out **why**.

Three things cause a fan-out

- ◆ **Genuinely one to many.** You joined a detail table for a filter and kept its rows.
- ◆ **Partial join key.** You joined on part but not on site. Half a key multiplies.
- ◆ **Source is not unique.** A text table with one row per language. A view with no dedup.
- ◆ The third one causes the most confusion, because the join looks correct.

THE POINT

Only the first is visible in the join clause. The other two need you to look at the data.

What you actually type

Name the join. Ask for proof, not explanation.

```
Row counts show the join to <table> triples the row count.  
The join condition is: <paste the ON clause>
```

Write three queries that will tell me which of these it is:

1. Is <table> unique on the join key? Group by the key, having count > 1, ordered by count descending, top 20.
2. Is the join key complete? Show me any column pair that appears in both tables but not in the ON clause.
3. Show me one example key with its duplicate rows in full.

Query three matters most. I want to see what differs between the duplicates, not be told what differs.

WHY QUERY THREE WINS

Seeing the duplicate rows side by side makes the cause obvious in seconds.

Two rows, one difference

Query three output for a single work order type.

```
01 wotyp    lang    wotypx
02 -----
03 C1       E       Corrective - Emergency
04 C1       F       Correctif - Urgence
05 C1       D       Korrektiv - Notfall
06
07 -- The ON clause joined on wotyp only.
08 -- The lookup holds one row per language.
09 -- Every work order was tripled. Hours tripled with it.
```

THE FINDING

The fix is four characters. Finding it without counts would have taken an afternoon.

Knowing when to stop asking and start again.

What happens across a long exchange

- ◆ Research on multi-turn SQL debugging finds quality degrades as the exchange goes on.
- ◆ Failed attempts stay in the context. They keep influencing what comes next.
- ◆ You start getting variations on earlier wrong answers, phrased more confidently.
- ◆ The fix is not a better follow-up. It is a clean start with what you learned.

THE EVIDENCE

This is measured behaviour, not folklore. Your fifteenth message is worse than your first.

How to recognise it happening

- ◆ The same suggestion returns, slightly reworded.
- ◆ Fixes start addressing symptoms you already ruled out.
- ◆ It apologises, then repeats the previous answer.
- ◆ Rough rule. Three failed attempts on one problem means restart.

THE HONEST PART

You will notice these signs and keep going anyway, because restarting feels like losing progress.

What you actually type

New session. This is the first message, not a follow-up.

I am debugging a query. Here is what is already established.

CONFIRMED BY QUERY:

- Join to t401 triples the row count
- t401 holds one row per language for each type code
- The ON clause does not filter language

RULED OUT BY QUERY:

- wopr join is one to one, count unchanged
- No duplicate work order numbers in the driving table

Given only that, write the corrected join clause and tell me what the row count should be after the fix.

THE HALF PEOPLE DROP

Both lists are findings. Carrying the second one is what stops the loop.

What you carry, what you leave

LEAVE BEHIND

The exchange

- Every failed attempt
- Every apology and revision
- Theories nobody tested
- The original query, if it is long

CARRY FORWARD

The findings

- Anything a query confirmed
- Anything a query ruled out
- The counts themselves
- The one question still open

THE TIEBREAKER

If a claim did not come from a query you ran, it does not go in the restart.

Counts cannot help you when the bug only
fires **some** nights.

Two clocks in one expression

Weekly snapshot. Runs nightly at 00:30 local.

```
01 v_snapshot_date INTEGER :=
02   CASE WHEN datepart('day', sysdate) = 1
03         THEN to_char(CONVERT_TIMEZONE('CST6CDT', SYSDATE-1), 'YYYYMMDD')
04         ELSE to_char(CONVERT_TIMEZONE('CST6CDT', SYSDATE), 'YYYYMMDD')
05   END;
06
07 -- The test uses SYSDATE. Raw. That is UTC.
08 -- The value uses the converted timestamp. That is local.
09 -- On the 1st, at 00:30 local, UTC already says the 1st.
10 -- Twelve times a year, the branch is chosen on the wrong day.
```

WHY NOBODY FOUND IT

Runs correctly on 353 nights. Picks the wrong branch on some of the other twelve.

Debugging something that is not there right now

- ◆ **Counts will not help.** Run them tonight and everything is correct.
- ◆ **Look for two sources of the same fact.** Two clocks, two calendars, two rounding rules.
- ◆ **Ask what input would make it differ.** That is the adversarial read, early.
- ◆ **Then check history.** If it fires monthly, the evidence is in the snapshot table.

THE POINT

The bug is not in the data you have. It is in the boundary between two ways of asking the same question.

The count triage shows the driving table at 890 thousand, unchanged after the first join, and 890 thousand after the second.

The final total is still thirty percent too high. Every join is one to one.

Where do you look next?

- ☐ A Re-run the triage. The counts must be wrong
- ☐ B The measure itself, not the joins
- ☐ C The source tables, for duplicate rows
- ☐ D The WHERE clause, for a filter that is too permissive

◆ PAUSE AND DECIDE

Where do you look next?

A Re-run the triage

The counts are simple and they agree with each other. Doubting your instrument first is rarely right.

C Duplicate source rows

That would have shown as a count increase. The triage already ruled it out.

B The measure itself

Row counts are correct, so the inflation is inside the calculation, not the row set.

D A permissive filter

A loose filter changes the count. The count is right, so this is not it.

THE SIGNAL

Correct row counts and a wrong total means the fault is in the arithmetic, not the join graph.

You have been debugging for forty minutes. The last three suggestions were variations on adding DISTINCT. You already tried that and it changed nothing.

You are fairly sure the problem is in the reservations join, but you have not proved it.

What do you do?

- ☐ A Tell it DISTINCT does not work and ask for something else
- ☐ B Restart with what you have confirmed and ruled out
- ☐ C Run the fan-out isolation queries on the reservations join first
- ☐ D Start a new session and paste the whole query again

◆ PAUSE AND DECIDE

What do you do?

- A

 Ask for something else
Three repeats is the signal to stop. A fourth request continues the loop you are already in.
- B

 Restart with your findings
Right instinct, and premature. Your best lead is a hunch, so the restart would carry a guess.
- C

 Run the isolation queries
Turns the hunch into a confirmed finding, which is what makes a restart worth doing.
- D

 New session, paste it all
A restart that carries no findings is just the same conversation with the evidence deleted.

THE SIGNAL

Restart when you have findings to carry. Get findings first, and the restart writes itself.

The triage shows the count dropping from 890 thousand to 604 thousand when a lookup table is joined. You were investigating a total that is too high.

The lookup is joined with an inner join to fetch a description.

What do you do with this?

- ☐ A Ignore it. You are looking for inflation, not loss
- ☐ B Note it and keep tracing the inflation
- ☐ C Stop and investigate the drop instead
- ☐ D Change it to a left join and re-run the triage

◆ PAUSE AND DECIDE

What do you do with this?

- A

 Ignore it
You just found a third of the rows disappearing into a lookup join. That is not a side issue.
- B

 Note it, keep going
Defensible and it defers the bigger finding. Nobody ever comes back to the noted item.
- C

 Investigate the drop
An inner join to a description table should not remove rows. Something is missing from that lookup.
- D

 Switch to left join
That restores the count and hides the cause. You still do not know why 286 thousand rows had no match.

THE SIGNAL

You were called about a number being too high. You just found a different number that is too low.

You find and fix the fan-out. The total drops to what the business expected. You are about to close the ticket.

The fix was a missing language filter in a join that has been in production for three years.

What else does this fix imply?

- ☐ A Nothing. The query is now correct
- ☐ B Every historical figure from this query was wrong too
- ☐ C The same lookup is probably joined elsewhere the same way
- ☐ D Both B and C, and both need saying out loud

◆ PAUSE AND DECIDE

What else does this fix imply?

- A

 Nothing further
The query is correct from tonight. Every number it produced before tonight was not.
- B

 History was wrong
True, and somebody made decisions on those numbers. It is half the answer.
- C

 It is joined elsewhere
Almost certainly, and one grep finds out. Also half the answer.
- D

 Both, said out loud
A three year old defect is never contained to the query where you happened to find it.

THE SIGNAL

The fix takes four characters. The conversation about three years of reported numbers takes longer.

Take this to work tomorrow

- ◆ Next wrong number, write the count triage query before you form a theory.
- ◆ When a count moves unexpectedly, ask for proof queries, not explanations.
- ◆ Keep a running list of confirmed and ruled out while you work.
- ◆ Three failed attempts on one problem. Restart. Set a timer if you have to.

THE PAYOFF

The confirmed and ruled out list costs nothing while you work and is the whole restart when you need it.

You never asked what was wrong. You asked for the **counts**.